

# Java Programming Puzzles — 50 Problems with Implementations

## 1. Reverse a number (no String)

Reverse digits of an integer without converting to String.

```
int n = 12345;
int rev = 0;
while (n > 0) {
    rev = rev * 10 + n % 10;
    n /= 10;
}
System.out.println(rev);
```

## 2. Count digits (no String)

Count number of digits in an integer.

```
int n = 98765;
int count = 0;
while (n > 0) { count++; n /= 10; }
System.out.println(count);
```

## 3. Swap two numbers without third variable

Swap a and b without using a temporary variable.

```
int a = 5, b = 7;
a = a + b;
b = a - b;
a = a - b;
// a=7, b=5
```

## 4. Missing number in 1..N

Find missing number from array containing 1..N with one missing.

```
int n = 5; int[] arr = {1,2,4,5};
int sum = n*(n+1)/2;
for (int x: arr) sum -= x;
System.out.println(sum);
```

## 5. First non-repeating character

Return first non-repeating character from a string.

```
String s = "stress";
Map<Character,Integer> map = new LinkedHashMap<>();
for(char c: s.toCharArray()) map.put(c, map.getOrDefault(c,0)+1);
for(char c: s.toCharArray()) if(map.get(c)==1){ System.out.println(c); break; }
```

## 6. Check anagrams

Check if two strings are anagrams.

```
char[] a1 = s1.toCharArray(); char[] a2 = s2.toCharArray();
Arrays.sort(a1); Arrays.sort(a2);
return Arrays.equals(a1,a2);
```

## 7. Second highest without sorting

Find second largest element in array without sorting.

```
int max1=Integer.MIN_VALUE, max2=Integer.MIN_VALUE;
for(int n: arr){ if(n>max1){ max2=max1; max1=n; } else if(n>max2 && n!=max1) max2=n; }
```

## 8. Detect loop in linked list (Floyd)

Detect cycle in linked list using fast and slow pointers.

```
ListNode slow=head, fast=head;
while(fast!=null && fast.next!=null){ slow=slow.next; fast=fast.next.next; if(slow==fast){ System.out.println("Cyc"
```

## 9. Palindrome number

Check if integer is palindrome without String.

```
int n=121; int orig=n, rev=0;
while(n>0){ rev=rev*10+n%10; n/=10; }
System.out.println(rev==orig);
```

## 10. Reverse Linked List

Reverse singly linked list in-place.

```
ListNode prev=null, curr=head;
while(curr!=null){ ListNode nxt=curr.next; curr.next=prev; prev=curr; curr=nxt; }
head=prev;
```

## 11. Merge two sorted arrays (in-place for larger array)

Merge sorted array B into A which has enough buffer at end.

```
int i=m-1, j=n-1, k=m+n-1;
while(j>=0) A[k--]=(i>=0 && A[i]>B[j])?A[i--]:B[j--];
```

## 12. Find duplicate using XOR

Find single duplicate where numbers 1..N-1 and one duplicate exist using XOR.

```
int xor=0; for(int i=1;i<=n-1;i++) xor ^= i; for(int v: arr) xor ^= v; System.out.println(xor);
```

## 13. Move zeros to end

Move zeros in array to the end while preserving order.

```
int idx=0;
for(int v: arr) if(v!=0) arr[idx++]=v;
while(idx<arr.length) arr[idx++]=0;
```

## 14. Rotate array by k (Juggling)

Rotate array to right by k positions using gcd method.

// Implementation involves juggling algorithm using gcd

## 15. Two sum (sorted array)

Find pair in sorted array that sums to target using two pointers.

```
int l=0, r=arr.length-1;
while(l<r){ int s=arr[l]+arr[r]; if(s==t) return new int[]{l,r}; if(s<t) l++; else r--; }
```

## 16. Implement stack using queues

Make LIFO using two FIFO queues.

// Push costly: enqueue to q2, move all from q1 to q2, swap names

## 17. Inorder traversal without recursion (stack)

Iterative inorder of binary tree using stack.

```
Stack<TreeNode> st=new Stack<>(); TreeNode cur=root;
while(cur!=null || !st.isEmpty()){ while(cur!=null){ st.push(cur); cur=cur.left; } cur=st.pop(); visit(cur); cur=cur.right; }
```

## 18. Validate parentheses

Check if string of parentheses is valid.

```
Map<Character,Character> m = Map.of('(',')', '(', ')', '[', ']', '{', '}');
Stack<Character> st=new Stack<>();
for(char c: s.toCharArray()){ if(m.containsKey(c)){ if(st.isEmpty()||st.pop()!=m.get(c)) return false; } else st.push(c); }
```

## 19. Fibonacci (iterative)

Compute nth Fibonacci iteratively (O(n) time, O(1) space).

```
long a=0, b=1;
for(int i=2; i<=n; i++){ long c=a+b; a=b; b=c; }
return (n==0)?0:b;
```

## 20. Binary search (iterative)

Classic binary search in sorted array.

```
int l=0,r=n-1; while(l<=r){ int m=(l+r)>>1; if(arr[m]==t) return m; if(arr[m]<t) l=m+1; else r=m-1; } return -1;
```

## 21. Count set bits (Brian Kernighan)

Count 1-bits in integer.

```
int count=0; while(n!=0){ n &= (n-1); count++; }
```

## 22. Power of two check

Check if n is power of two.

```
boolean isPow = n>0 && (n & (n-1))==0;
```

## 23. Reverse words in sentence

Reverse word order in a sentence.

```
String[] parts = s.trim().split("\\s+"); Collections.reverse(Arrays.asList(parts)); return String.join(" ", parts);
```

## 24. Longest common prefix

Find LCP among array of strings.

```
String prefix = strs[0]; for(String s: strs) while(!s.startsWith(prefix)) prefix = prefix.substring(0,prefix.length-1);
```

## 25. Matrix transpose (in-place square)

Transpose square matrix in-place.

```
for(int i=0;i<n;i++) for(int j=i+1;j<n;j++){ int t=a[i][j]; a[i][j]=a[j][i]; a[j][i]=t; }
```

## 26. Spiral matrix traversal

Print matrix in spiral order.

```
// Implementation uses boundaries: top,bottom,left,right and loops
```

## 27. Find Kth smallest using QuickSelect

Select kth smallest element average O(n).

```
// Implement Lomuto partition and recurse on correct side
```

## 28. Serialize/deserialize binary tree (preorder)

Convert tree to string and back with null markers.

```
// Use DFS with commas and '#' for nulls
```

## 29. Check if two trees are identical

Compare structure and values recursively.

```
boolean same(TreeNode a, TreeNode b){ if(a==null&&b==null) return true; if(a==null||b==null) return false; return a.val==b.val && same(a.left,b.left) && same(a.right,b.right); }
```

## 30. Implement Trie (insert/search)

Prefix tree for words with insert, search, startsWith.

```
// Node with children[26] and isEnd flag; straightforward methods
```

## 31. LRU Cache (LinkedHashMap)

LRU cache with O(1) get/put.

```
class LRU<K,V> extends LinkedHashMap<K,V>{ private int cap; LRU(int c){ super(c,0.75f,true); cap=c; } protected boolean removeEldestEntry(Map.Entry<K,V> eldest){ return eldest.getValue().size()>cap; }
```

## 32. Min stack (O(1) getMin)

Stack that returns minimum in O(1).

```
Stack<Integer> s=new Stack<>(); Stack<Integer> min=new Stack<>();
void push(int x){ s.push(x); if(min.isEmpty()||x<=min.peek()) min.push(x); }
void pop(){ if(s.pop().equals(min.peek())) min.pop(); }
```

### 33. Merge intervals

Merge overlapping intervals.

```
Arrays.sort(intervals, Comparator.comparingInt(a->a[0])); List<int[]> res=new ArrayList<>(); for(int[] in: intervals)
```

### 34. Kth largest (heap)

Find kth largest using a min-heap of size k.

```
PriorityQueue<Integer> pq=new PriorityQueue<>(); for(int v:arr){ pq.add(v); if(pq.size()>k) pq.poll(); } return pq
```

### 35. Topological sort (Kahn)

Topo order for DAG using in-degrees.

```
// Compute indegree, queue nodes with 0 indegree, pop and reduce neighbors
```

### 36. Word search in grid (DFS backtracking)

Check if word exists in 2D board.

```
// Backtrack with visited marking and 4-direction moves
```

### 37. Subset generation (backtracking)

Generate all subsets (power set).

```
void backtrack(int i, List<Integer> cur){ if(i==n){ res.add(new ArrayList<>(cur)); return; } cur.add(a[i]); backtrack
```

### 38. Combination Sum (unique combinations)

Find combinations summing to target using candidates.

```
// DFS with pruning and skipping duplicates
```

### 39. Count islands (DFS/BFS)

Count connected components of 1s in grid.

```
for(i,j) if(grid[i][j]=='1'){ dfs(i,j); count++; } // dfs marks visited
```

### 40. Serialize graph (adjacency list)

Convert graph to string and back.

```
// Use node ids and list commas; careful with disconnected nodes
```

### 41. Implement FutureTask-like (callable with thread)

Wrap Callable and provide get() that blocks till result.

```
// Use Thread to run callable and synchronized wait/notify for result
```

### 42. Dining philosophers (simple solution)

Avoid deadlock by picking lower-numbered fork first.

```
// Number forks and acquire in consistent order to avoid cycle
```

### 43. Producer-consumer (wait/notify)

Classic bounded buffer with wait/notify.

```
synchronized void put(E e){ while(full) wait(); enqueue(e); notifyAll(); } synchronized E take(){ while(empty) wait
```

### 44. Thread-safe singleton (double-checked locking)

Create lazy singleton with DCL.

```
private static volatile Singleton inst;
```

```
public static Singleton get(){ if(inst==null){ synchronized(Singleton.class){ if(inst==null) inst=new Singleton();
```

#### **45. Count inversions (merge sort)**

Count pair  $ia[j]$  via modified merge sort.

// During merge, when right element moved before left, add remaining left count

#### **46. Reservoir sampling**

Randomly pick  $k$  items from large/unknown stream.

// Fill first  $k$ , then for  $i \geq k$  replace with prob  $k/(i+1)$

#### **47. Find median of two sorted arrays (log min(m,n))**

Complex algorithm using binary search on partitions.

// Partition smaller array and ensure left max  $\leq$  right min across halves

#### **48. Regular expression matching ('.' and '\*')**

Match text to pattern with '.' and '\*' using DP.

// DP table  $dp[i][j]$  = ... typical implementation

#### **49. Edit distance (Levenshtein)**

Compute min edits between two strings using DP.

$int[][] dp = new int[m+1][n+1];$  // fill base cases and transitions

#### **50. N-Queens (backtracking)**

Place  $N$  queens so no two attack each other.

$void solve(int r){ if(r==n){ addSolution(); return; } for(int c=0;c<n;c++) if(ok(r,c)){ place; solve(r+1); remove;$